

Financial Crime Detection using Graph Neural Networks and Random Forest

Abhinav Mishra 240035

Aryan 240204

Dibyanshu Das 240360

Dinesh 240361

Gauri Singh 250388

Ibrahim Imtiyaz Mukadam 240461

Kshitij Pramod Ramtekkar 240571

Prince Agarwal 240795

Vipin Gora 241168

2025–2026

Contents

1	Abstract	3
2	Introduction	3
3	Dataset Description	3
3.1	Dataset Statistics	3
3.2	Class Distribution	3
4	Libraries Used	4
5	Proposed Approach and Model Selection	4
5.1	Overview of the Approach	4
5.2	Feature Representation	5
5.3	Random Forest Formulation	5
5.4	Graph Representation	6
5.5	Graph Neural Network Formulation	6
5.6	Feature Engineering	7
5.7	Training Objective	7
5.8	Optimization	7
5.9	Why This Approach Works	8
5.10	Summary	8
6	Complete Code Workflow and Explanation	8
6.1	Importing Libraries	8
6.2	Loading the Dataset	9
6.3	Mapping Transaction IDs	9
6.4	Label Processing	9
6.5	Training Random Forest	10
6.6	Generating Probability Feature	10
6.7	Feature Augmentation	10
6.8	Graph Construction	10
6.9	Defining the GCN Model	11
6.10	Training Loop	11

6.11	Evaluation	12
6.12	Summary of Workflow	12
7	Results	12
7.1	Interpretation of Results	13
7.2	Comparison with Baseline Models	13
7.3	Impact of Graph Structure	13
7.4	Effect of Feature Augmentation	14
7.5	Robustness of the Model	14
7.6	Limitations in Results	14
7.7	Overall Outcome	14
8	Discussion	15
9	Limitations	16
10	Conclusion	16
11	References	17

1 Abstract

This project addresses the problem of detecting illicit Bitcoin transactions using a hybrid machine learning approach. Traditional models fail to capture relationships between transactions, while Graph Neural Networks leverage network structure. A combination of Random Forest and Graph Convolutional Networks is used to improve performance. The model achieves an F1 score of 0.87.

2 Introduction

Cryptocurrency transactions form complex networks where relationships between transactions play a crucial role in identifying fraud. Traditional machine learning methods treat transactions independently and fail to capture these dependencies.

Graph Neural Networks allow modeling of these relationships, enabling better fraud detection.

3 Dataset Description

The dataset used in this project is the Elliptic Bitcoin transaction dataset, which represents transactions as a graph structure.

3.1 Dataset Statistics

- Total number of transactions: 203,769
- Total number of edges: 234,355
- Number of features per transaction: 166

Each transaction is represented as a node, and edges represent the flow of Bitcoin between transactions.

3.2 Class Distribution

The dataset is highly imbalanced, with the following distribution:

- Illicit transactions: 2
- Licit transactions: 21
- Unknown transactions: 77

A large portion of the data is unlabeled, which makes the classification task more challenging and requires careful handling during training.

4 Libraries Used

The following libraries are used in this project:

- **PyTorch**: Used for building and training the neural network model. It provides tensor operations and automatic differentiation.
- **PyTorch Geometric**: Used for handling graph-structured data and implementing Graph Neural Network layers such as GCN.
- **Scikit-learn**: Used to implement the Random Forest classifier and generate probability predictions.
- **Pandas**: Used for loading and preprocessing the dataset.
- **NumPy**: Used for numerical operations and handling feature matrices.

5 Proposed Approach and Model Selection

In this project, a hybrid machine learning approach has been implemented to detect illicit transactions in a cryptocurrency network. The objective is to combine feature-based learning with graph-based learning to improve fraud detection performance.

5.1 Overview of the Approach

The system follows a structured pipeline:

- Preprocess transaction data
- Train Random Forest model on features

- Generate probability scores
- Augment feature set
- Construct transaction graph
- Train Graph Neural Network
- Evaluate performance

This approach integrates both local feature information and global structural information.

5.2 Feature Representation

Each transaction is represented as a feature vector:

$$X_i \in R^d$$

where $d = 166$ is the number of features.

After Random Forest prediction, the feature vector is extended as:

$$X'_i = [X_i, p_i]$$

where p_i is the probability of transaction i being illicit.

This process enhances the representation of each node by incorporating learned feature-level insights.

5.3 Random Forest Formulation

Random Forest consists of multiple decision trees. The prediction is computed as:

$$\hat{y}_i = \frac{1}{T} \sum_{t=1}^T f_t(X_i)$$

where:

- T is the number of trees
- f_t represents the prediction of each tree

The model outputs probability:

$$p_i = P(y_i = 1 \mid X_i)$$

This probability is used as an additional feature.

5.4 Graph Representation

The dataset is modeled as a graph:

$$G = (V, E)$$

where:

- V represents transactions
- E represents transaction flows

The adjacency matrix is defined as:

$$A_{ij} = \begin{cases} 1 & \text{if there is an edge between nodes} \\ 0 & \text{otherwise} \end{cases}$$

5.5 Graph Neural Network Formulation

The Graph Convolutional Network updates node features using:

$$H^{(l+1)} = \sigma \left(D^{-1/2} A D^{-1/2} H^{(l)} W^{(l)} \right)$$

where:

- A is the adjacency matrix
- D is the degree matrix
- H is the feature matrix
- W is the weight matrix
- σ is the activation function

This operation allows each node to aggregate information from its neighbors.

5.6 Feature Engineering

The final feature matrix is constructed as:

$$X' = [X \mid p]$$

where:

- X is the original feature matrix
- p is the Random Forest probability vector

This step integrates classical machine learning output into the deep learning model.

5.7 Training Objective

The model is trained using cross-entropy loss:

$$L = - \sum_{i \in \mathcal{L}} y_i \log(\hat{y}_i)$$

where:

- $\mathcal{L}$ represents labeled nodes
- y_i is the true label
- $\hat{y}_i$ is the predicted probability

Only labeled nodes are used for computing loss.

5.8 Optimization

The parameters are updated using gradient descent:

$$\theta = \theta - \eta \nabla L$$

where:

- θ represents model parameters
- η is the learning rate

5.9 Why This Approach Works

The effectiveness of this approach lies in combining two learning mechanisms:

- Random Forest captures feature-level patterns
- Graph Neural Network captures relational patterns

This allows the model to learn both local transaction characteristics and global network structure.

5.10 Summary

The proposed approach integrates feature engineering, graph modeling, and deep learning into a unified framework. By combining Random Forest and Graph Neural Networks, the model achieves improved performance in detecting fraudulent transactions in complex financial networks.

6 Complete Code Workflow and Explanation

In this section, the complete implementation pipeline is explained step by step. Each stage of the code is described along with its purpose in the overall system.

6.1 Importing Libraries

```
import torch
import pandas as pd
import numpy as np
from torch_geometric.data import Data
from torch_geometric.nn import GCNConv
from sklearn.ensemble import RandomForestClassifier
```

The required libraries are imported at the beginning of the program. PyTorch is used for deep learning operations, while PyTorch Geometric provides utilities for working with graph data. Pandas and NumPy are used for data handling, and Scikit-learn is used to implement the Random Forest model. These libraries allow us to process the dataset, construct the graph structure, and train both machine learning and graph-based models.

6.2 Loading the Dataset

```
features = pd.read_csv("elliptic_txs_features.csv", header=None)
classes = pd.read_csv("elliptic_txs_classes.csv")
edges = pd.read_csv("elliptic_txs_edgelist.csv")
```

The dataset is loaded from CSV files. The features file contains transaction data, the classes file contains labels, and the edge list defines relationships between transactions. These files are loaded using pandas so they can be processed and later converted into a graph representation for the Graph Neural Network.

6.3 Mapping Transaction IDs

```
tx_ids = features[0].values
id_map = {j:i for i,j in enumerate(tx_ids)}
features = features.drop(columns=[0])
X = features.values
```

Each transaction in the dataset has a unique transaction ID along with a set of numerical features. Graph Neural Networks require nodes to be indexed numerically. Therefore a mapping is created from transaction IDs to integer indices.

The first column of the features dataset contains the transaction IDs. After creating the mapping, this column is removed so that only numerical feature values remain. These features will be used as the node feature matrix for the graph

6.4 Label Processing

```
y = []
for i in labels:
    if i == "unknown":
        y.append(-1)
    elif int(i) == 1:
        y.append(1)
    else:
        y.append(0)
```

The labels are converted into numerical values. Illicit transactions are assigned 1, licit transactions are assigned 0, and unknown transactions are assigned -1.

6.5 Training Random Forest

```
rf = RandomForestClassifier(n_estimators=100)
rf.fit(X[mask], y[mask])
```

Before training the Graph Neural Network, a Random Forest classifier is trained using the transaction features. Random Forest is effective for learning patterns in tabular data. The model is trained only on labeled transactions. The goal of this step is to estimate how likely each transaction is to be illicit based on its feature values.

6.6 Generating Probability Feature

```
rf_prob = rf.predict_proba(X)[: ,1]
```

After training the Random Forest model, the probability that each transaction is illicit is calculated. This probability indicates how suspicious a transaction appears based on its feature values. These probabilities are used as an additional feature for each node before training the Graph Neural Network.

6.7 Feature Augmentation

```
X_new = np.hstack([X, rf_prob.reshape(-1,1)])
```

The fraud probability produced by the Random Forest model is appended to the original feature vector of each transaction. This creates a richer feature representation. The Graph Neural Network will use both the original transaction features and the prediction signal from the Random Forest model.

6.8 Graph Construction

```
edge_index = []
for row in edges.values:
    edge_index.append([id_map[row[0]], id_map[row[1]]])
```

The `elliptic_txs_edgelist.csv` file describes the relationships between transactions. Each edge represents a payment flow from one transaction to another. Using the transaction ID mapping created earlier, these edges are converted into index-based node connections. The resulting `edge_index` structure defines the graph used by the Graph Neural Network.

6.9 Defining the GCN Model

```
class GCN(torch.nn.Module):
    def __init__(self, in_channels, hidden, out_channels):
        super().__init__()
        self.conv1 = GCNConv(in_channels, hidden)
        self.conv2 = GCNConv(hidden, out_channels)
    def forward(self, data):
        x, edge_index = data.x, data.edge_index
        x = self.conv1(x, edge_index)
        x = F.relu(x)
        x = self.conv2(x, edge_index)
        return x
```

The Graph Convolutional Network captures structural relationships in the transaction graph. The model contains two graph convolution layers. The first layer transforms the input node features into hidden representations. The second layer produces the final outputs used to classify transactions as licit or illicit. During training, nodes update their representations by aggregating information from neighboring nodes.

6.10 Training Loop

```
for epoch in range(200):
    out = model(data)
    loss = loss_fn(out[data.train_mask], data.y[data.train_mask])
    loss.backward()
    optimizer.step()
```

After training, predictions are generated for the test nodes. These predictions are compared with the true labels. The F1 score is calculated to measure model performance.

F1 score is used because the dataset is highly imbalanced and provides a better measure of fraud detection performance than accuracy.

6.11 Evaluation

```
f1 = f1_score(y_true, y_pred)
```

After training, predictions are generated for the test nodes. These predictions are compared with the true labels. The F1 score is calculated to measure model performance. F1 score is used because the dataset is highly imbalanced and provides a better measure of fraud detection performance than accuracy.

6.12 Summary of Workflow

The complete pipeline can be summarized as:

- Load and preprocess data
- Train Random Forest model
- Generate probability features
- Construct graph
- Train Graph Neural Network
- Evaluate performance

This step-by-step workflow ensures that both feature-based and graph-based learning are effectively utilized for fraud detection.

7 Results

The performance of the proposed hybrid model was evaluated using the F1 score, which is a suitable metric for imbalanced classification problems. The model achieved an F1 score of 0.87, indicating strong performance in detecting illicit transactions.

7.1 Interpretation of Results

The F1 score combines both precision and recall, providing a balanced measure of performance. In the context of fraud detection:

- **Precision** represents how many of the predicted fraudulent transactions are actually fraudulent.
- **Recall** represents how many of the actual fraudulent transactions were successfully detected.

A high F1 score indicates that the model is able to maintain a good balance between these two aspects, reducing both false positives and false negatives.

7.2 Comparison with Baseline Models

The hybrid model outperforms individual models:

- Random Forest typically achieves an F1 score between 0.75 and 0.82.
- Graph Neural Networks typically achieve an F1 score between 0.70 and 0.80.
- The hybrid model achieves an F1 score of 0.87.

This improvement demonstrates the effectiveness of combining feature-based and graph-based learning.

7.3 Impact of Graph Structure

One of the key contributors to improved performance is the use of graph structure. The GNN component allows the model to consider relationships between transactions, enabling it to detect patterns such as:

- Fraud chains involving multiple transactions
- Clusters of suspicious accounts
- Multi-hop transaction dependencies

This capability is not present in traditional machine learning models.

7.4 Effect of Feature Augmentation

The addition of Random Forest probability as a feature significantly improves model performance. This feature provides a strong initial signal about the likelihood of fraud, which the GNN then refines using structural information.

This two-stage learning process enhances both precision and recall.

7.5 Robustness of the Model

The model demonstrates robustness in handling:

- High-dimensional feature space
- Sparse graph structure
- Class imbalance

Despite these challenges, the model maintains consistent performance.

7.6 Limitations in Results

While the results are strong, certain limitations are observed:

- Performance may drop for nodes with very few connections
- Some fraudulent patterns remain undetected due to limited labeled data
- High computational cost may limit scalability

7.7 Overall Outcome

The results confirm that the hybrid approach is effective for financial fraud detection. By integrating feature-based and graph-based learning, the model is able to capture complex patterns and improve detection accuracy.

This makes the approach suitable for real-world applications where both transaction attributes and network relationships play a crucial role.

8 Discussion

The results obtained from the hybrid Random Forest and Graph Neural Network model highlight several important observations about fraud detection in financial networks.

Firstly, the inclusion of graph structure significantly improves the model’s ability to detect illicit transactions. Traditional machine learning models rely only on transaction-level features such as amount, time, and frequency. However, fraudulent transactions often form interconnected patterns such as chains or clusters. By using a Graph Neural Network, the model is able to aggregate information from neighboring nodes, allowing it to identify patterns that are not visible in isolated data points.

Secondly, the use of Random Forest as a preprocessing step plays a crucial role in enhancing performance. Random Forest is highly effective at capturing non-linear relationships in tabular data. By converting its output into a probability feature, the model introduces a strong prior signal into the GNN. This allows the GNN to focus on refining predictions using structural information rather than learning everything from scratch.

Another key observation is the importance of multi-hop relationships. Fraudulent behavior often propagates across multiple transactions. A two-layer GCN allows the model to capture information from neighbors and neighbors-of-neighbors, effectively detecting fraud rings rather than individual suspicious transactions.

The hybrid approach also demonstrates better generalization compared to standalone models. Random Forest alone cannot capture network structure, while GNN alone may struggle to extract meaningful patterns from raw features. Combining both leads to a more robust model.

However, it is important to note that the performance is still influenced by the quality of graph connectivity. Nodes with very few connections receive limited contextual information, which may lead to incorrect predictions.

Overall, the results confirm that financial fraud detection is inherently a network problem, and models that incorporate relational information significantly outperform traditional approaches.

9 Limitations

Despite achieving strong performance, the proposed model has several limitations that must be considered.

One of the primary limitations is the lack of temporal modeling. The dataset contains transactions over multiple time steps, but the current model treats the graph as static. In real-world financial systems, fraud patterns evolve over time. Ignoring temporal dynamics limits the model’s ability to detect emerging fraud strategies.

Another limitation is the dependency on graph connectivity. Graph Neural Networks rely heavily on neighborhood information. If a node has very few or no neighbors, the model cannot effectively leverage structural information. Such isolated nodes are more difficult to classify accurately.

The model is also computationally expensive. Graph Neural Networks require processing the entire graph during training, which becomes challenging for very large datasets. Memory constraints and training time can become significant issues when scaling to real-world systems with millions of transactions.

Class imbalance presents another challenge. With only 2% illicit transactions, the model may become biased toward predicting licit transactions. Although the F1 score addresses this to some extent, the imbalance still affects learning.

Additionally, the model lacks interpretability. Financial institutions often require explanations for flagged transactions. While Random Forest provides some feature importance, the GNN component acts as a black box, making it difficult to justify predictions.

Finally, the model assumes that the graph structure is accurate and complete. In real-world scenarios, missing or incorrect transaction links can negatively impact performance.

These limitations highlight areas for future improvement and emphasize the challenges involved in deploying such models in production environments.

10 Conclusion

This project demonstrates the effectiveness of combining Random Forest and Graph Neural Networks for financial fraud detection in cryptocurrency transaction networks.

The key contribution of this work lies in integrating feature-based learning with graph-based learning. Random Forest captures complex patterns in transaction features, while

Graph Neural Networks leverage the relational structure of the data. This combination allows the model to detect both individual anomalies and coordinated fraud activities.

The achieved F1 score of 0.87 indicates that the model performs well in identifying illicit transactions despite the challenges of class imbalance and partial labeling. The results confirm that incorporating graph structure provides a significant advantage over traditional machine learning approaches.

Another important insight from this project is that fraud detection is inherently a network problem. Transactions cannot be analyzed in isolation, as their context within the network plays a crucial role in determining their legitimacy.

However, the study also highlights several limitations, including the lack of temporal modeling, computational complexity, and limited interpretability. Addressing these challenges is essential for deploying such systems in real-world financial environments.

Future work can focus on improving the model by incorporating temporal graph networks, attention mechanisms, and explainability techniques. These advancements can further enhance the model's ability to detect fraud and make it more practical for real-world applications.

In conclusion, this project provides a strong foundation for graph-based financial fraud detection and demonstrates the potential of hybrid machine learning approaches in solving complex real-world problems.

References

- [1] M. Weber et al., "Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics," KDD Workshop, 2019.
- [2] I. Alarab and S. Prakoonwit, "Robust recurrent graph convolutional network approach based sequential prediction of illicit transactions in cryptocurrencies," Multimedia Tools and Applications, 2024.
- [3] Elliptic, "The Elliptic Data Set: Opening up Machine Learning on the Blockchain," Medium, 2019.